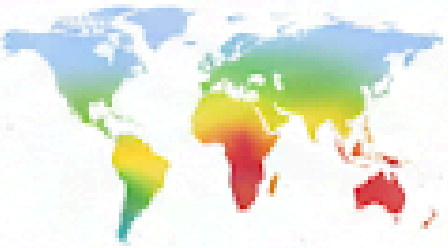
Pearls AQI Predictor

End-to-End Serverless MLOps Pipeline
for AQI Forecasting



Author: Aroon Kumar

Document Overview

About This Document

This document presents the complete technical architecture, design decisions, and MLOps implementation of the **Pearls AQI Predictor** system — an end-to-end serverless Machine Learning pipeline developed to forecast Air Quality Index (AQI) for the next three days using real-time environmental and weather data.

The purpose of this document is to provide a detailed explanation of how raw environmental signals are transformed into reliable, automated, and explainable AQI forecasts using modern MLOps practices. This report covers the full lifecycle of the system including data ingestion, feature engineering, feature store integration, historical data backfilling, model training, CI/CD orchestration, model serving, and visualization.

This document is written in the style of production ML system documentation and reflects real industry practices used in modern ML platforms.

Project Objectives

The key objectives of the Pearls AQI Predictor system are:

- To forecast AQI for the next 72 hours using real-time data
- To design a reproducible and automated MLOps pipeline
- To ensure training–serving feature parity using a Feature Store
- To implement serverless orchestration using GitHub Actions
- To provide explainable predictions using SHAP
- To visualize predictions and trends through an interactive dashboard

---

Scope of the System

This project covers:

- Real-time data ingestion from AQICN and OpenWeather APIs
- Advanced feature engineering for time-series AQI prediction
- Historical dataset creation through feature pipeline replay

- Training multiple ML models and selecting the best automatically
- Maintaining a model registry and automated deployment
- Serving predictions via Flask API
- Visualizing results and explanations through Streamlit
- Fully automated CI/CD workflows without managing servers

Intended Audience

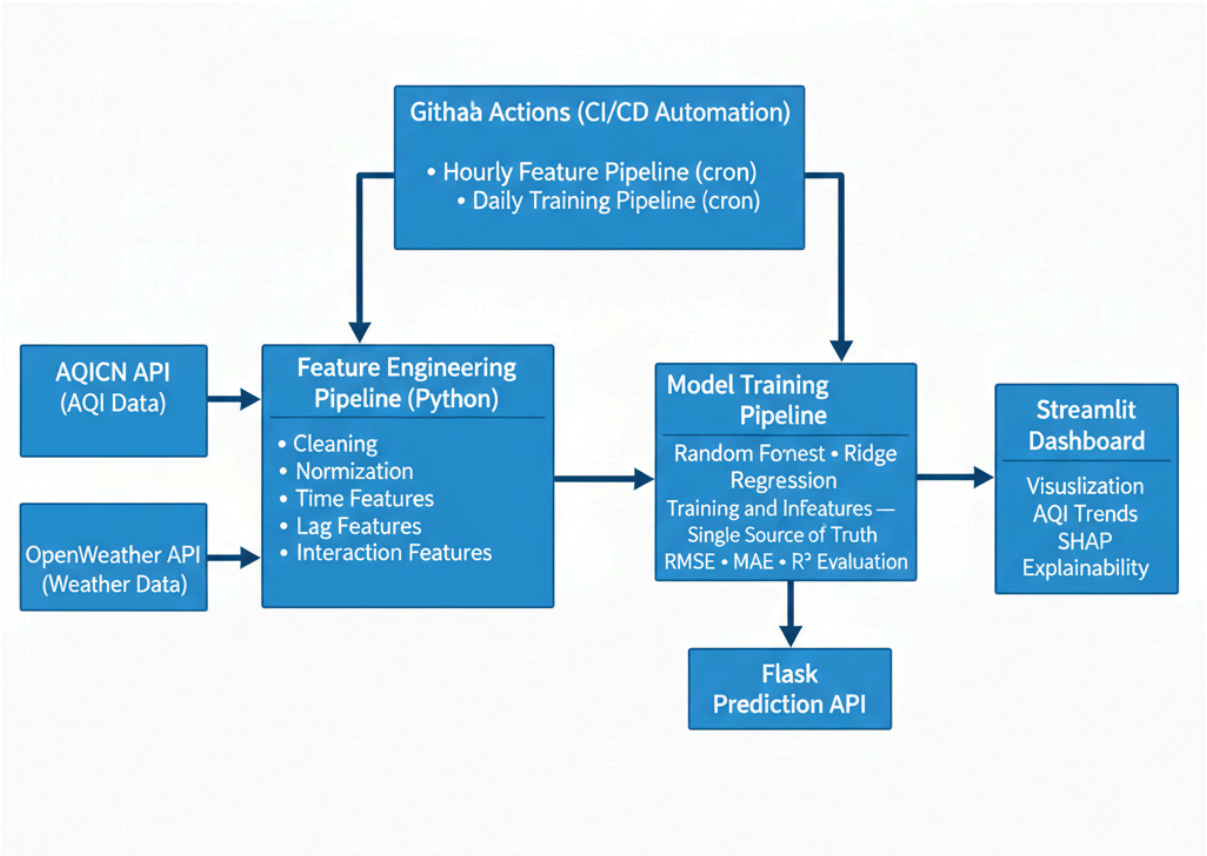
This document is intended for:

- ML Engineers and MLOps Engineers
- Data Engineers
- Software Engineers working on ML systems
- Technical reviewers and architects
- Anyone interested in production-grade ML system design

Technology Stack Used

Category	Tools & Technologies
	Python
Programming	Scikit-learn,
ML Libraries	TeneSFrowr
Feature Store	Hopsworks API, OpenWeather API
Data APIs	CI/CD
Dashbard	Flask
Explainability	SHAP

3. System Architecture



The Pearls AQI Predictor follows a modular, production-grade MLOps architecture designed to ensure automation, reproducibility, and scalability without managing servers.

The system begins with real-time data ingestion from two external sources:

- **AQICN API** for pollutant and AQI readings
- **OpenWeather API** for weather parameters such as temperature, humidity, wind speed, and pressure

This data flows into the **Feature Engineering Pipeline**, where raw responses are cleaned, normalized, and transformed into predictive features. These engineered features are stored in a **Feature Store (Hopsworks / Vertex AI)** which acts as the single source of truth for both training and inference.

A **Training Pipeline** runs daily, consuming historical features from the feature store to train multiple ML models. The best model is automatically saved into the **Model Registry**.

For inference, a **Flask API** loads the latest registered model and fetches the latest features from the feature store to generate predictions. These predictions are displayed through a **Streamlit Dashboard**.

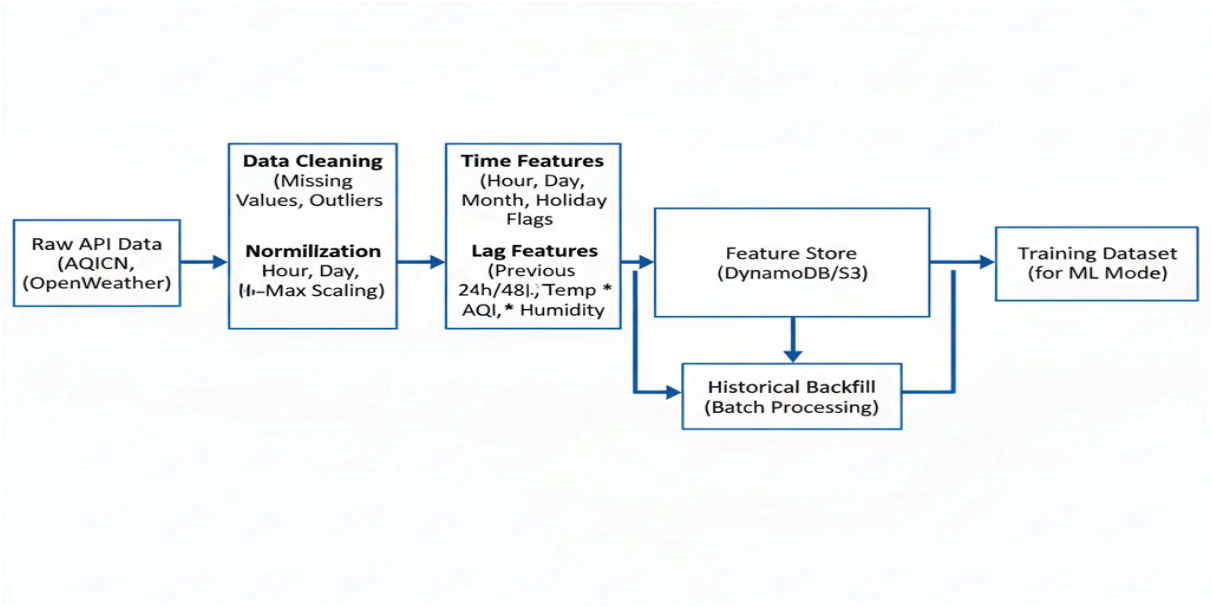
All workflows are orchestrated using **GitHub Actions**, which trigger:

- Hourly feature ingestion jobs
- Daily model training jobs

This architecture ensures the system is fully automated and serverless.

---

4. Feature Engineering Pipeline



Feature engineering is the core strength of this project. Instead of relying on raw AQI and weather values, the system creates rich time-series features that allow models to understand AQI behavior.

The pipeline performs:

Data Cleaning and Normalization

- Schema alignment between AQICN and OpenWeather
- Handling missing and null values
- Unit normalization

Time-Based Features

- Hour of day
- Day of week
- Month
- Weekend/weekday indicator

Lag Features

- AQI from previous 1, 3, 6, 12, and 24 hours
- Pollutant lag values

Rolling Statistics

- Moving averages
- AQI change rate
- Trend indicators

Interaction Features

- Pollutant × humidity
- Temperature × pressure
- Wind speed × PM2.5

These features are stored in the feature store and used consistently for both training and prediction.

---

5. Historical Data Backfill

A major challenge was the absence of historical datasets matching the engineered feature schema.

To solve this, the system replays the entire feature pipeline on past timestamps:

1. Iterate over past dates and times
2. Call APIs with historical parameters
3. Pass data through the same feature engineering steps
4. Store features into the feature store

This guarantees **training–serving parity**. The model never sees features during inference that it did not see during training.

This approach ensures reproducibility and allows retraining anytime with consistent data.

---

6. Training Pipeline

The training pipeline is executed daily through GitHub Actions.

Steps include:

- Fetch historical features from feature store
- Perform time-aware train/validation split
- Train multiple models:
    - Random Forest Regressor
    - Ridge Regression
    - TensorFlow Neural Network
- Evaluate using RMSE, MAE, and R^2
- Perform cross-validation across time windows
- Automatically register the best model

Each training run is versioned with:

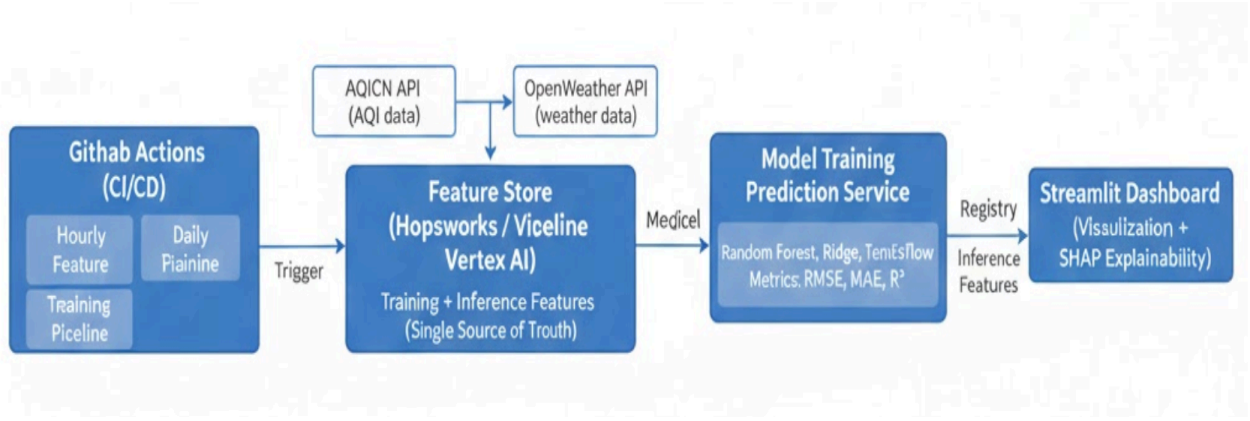
- Feature version
- Training timestamp

- Model metrics

This makes the pipeline fully reproducible.

---

7. CI/CD and Automation



GitHub Actions acts as the serverless orchestrator for the entire system.

Hourly Workflow

- Trigger feature ingestion
- Update feature store with latest data

Daily Workflow

- Trigger training pipeline
- Update model registry

The Flask API always loads the latest model from the registry. No manual deployment is required.

This creates a **self-updating ML system**.

---

8. Web Dashboard and Prediction Service

The system uses a two-layer serving architecture.

Flask API

- Loads latest model
- Fetches latest features
- Produces 3-day AQI forecast
- Exposes REST endpoints

Streamlit Dashboard

- Displays current AQI
- Shows forecasts and trends
- Displays SHAP explainability
- Alerts when AQI exceeds thresholds

---

9. Explainability with SHAP

SHAP is integrated to explain every prediction.

It shows:

- Which pollutant contributed most
- How weather conditions influenced AQI
- How historical AQI affected the forecast

This builds trust and provides environmental insights beyond prediction.

---

10. Challenges Faced and Solutions

Challenge	Solution	Learning
Inconsistent API formats	Schema normalization	Standardize external data
Missing API values	Imputation & retries	Build resilient pipelines
Timestamp mismatch	UTC alignment	Time consistency is critical
Feature store schema	Versioned design	Schema defines scalability
No historical dataset	Feature pipeline replay	Backfill is superior
Model instability	Multiple models & CV	Ensemble improves reliability
Data drift	Daily retraining	Automation handles drift
API rate limits	Caching & batching	Respect API limits
Reproducibility	CI/CD training	Automation ensures consistency
Model deployment	Model registry	Registry is essential
No servers	GitHub Actions	CI tools can replace Airflow

---

11. Key Learnings

- Feature engineering matters more than model complexity
- Feature store is the backbone of MLOps
- Training-serving parity prevents hidden bugs
- CI/CD is mandatory for ML systems
- Explainability increases trust
- Serverless orchestration is powerful and cost-effective

---

12. Conclusion

Pearls AQI Predictor demonstrates how data engineering, machine learning, DevOps automation, and explainability integrate into a real production-grade MLOps system. From data ingestion to real-time serving, the system reflects industry practices used in modern ML platforms and showcases strong MLOps and ML system design expertise.